POLITECNICO
MILANO 1863

Credit Card Customer Attrition

FinTech Project Report

Arnaud Grassian

Politecnico di Milano

June 2026

Contents

1 Introduction

This project follows three modelling approaches : a logistic regression with L1 regularization (Lasso) followed by a Random Forest on the surviving features, a Random Forest trained directly on the full feature set, and an XGBoost classifier. All three are tuned consistently on the same training data, with the same cross-validation strategy and the same scoring metric.

Beyond the standard pipeline, I add two contributions specific to this project :

- A business cost function that puts a dollar value on false positives (sending a retention offer to a customer who would have stayed anyway) and false negatives (missing a customer who eventually leaves). I tune the decision threshold to maximize expected profit instead of F1, which gives a more useful recommendation for the bank.
- A leakage-safe second test where I drop the features that describe the customer's behaviour during the churn process itself, and add three rolling-window ratios in their place. This shows how much performance is lost when the model is restricted to features that can be updated monthly in production.

This report describes the methodology, the implementation, the results, and the limits of the analysis.

2 Exploratory Data Analysis

2.1 Dataset overview

The dataset contains 10 127 rows and 21 columns. The target variable `Attrition_Flag` takes two values : Existing Customer (84%) and Attrited Customer (16%).

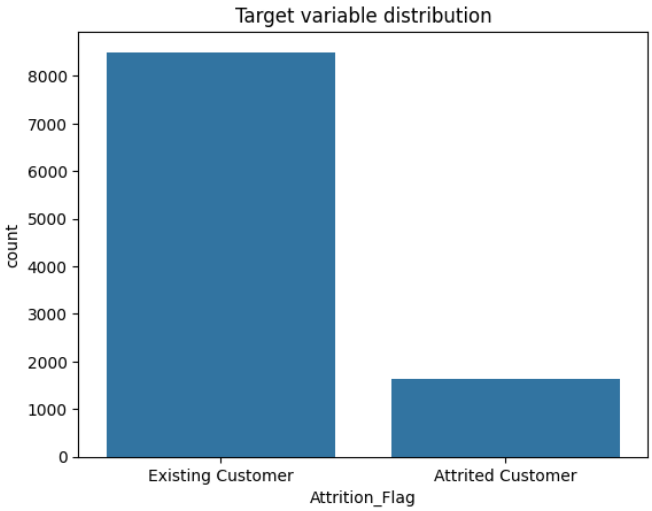

Figure 1: Distribution of the target variable. The class imbalance is moderate but not extreme : 16% of customers have closed their card.

The 21 features split into 14 numerical and 5 categorical variables. The numerical features describe transaction behaviour (counts, amounts, change ratios between Q4 and Q1), account characteristics (credit limit, revolving balance, utilization ratio), customer profile (age, dependent count, months on book), and contact history (months inactive, contacts in the last 12 months). The categorical features cover gender, education level, marital status, income category, and card category.

2.2 Numerical features

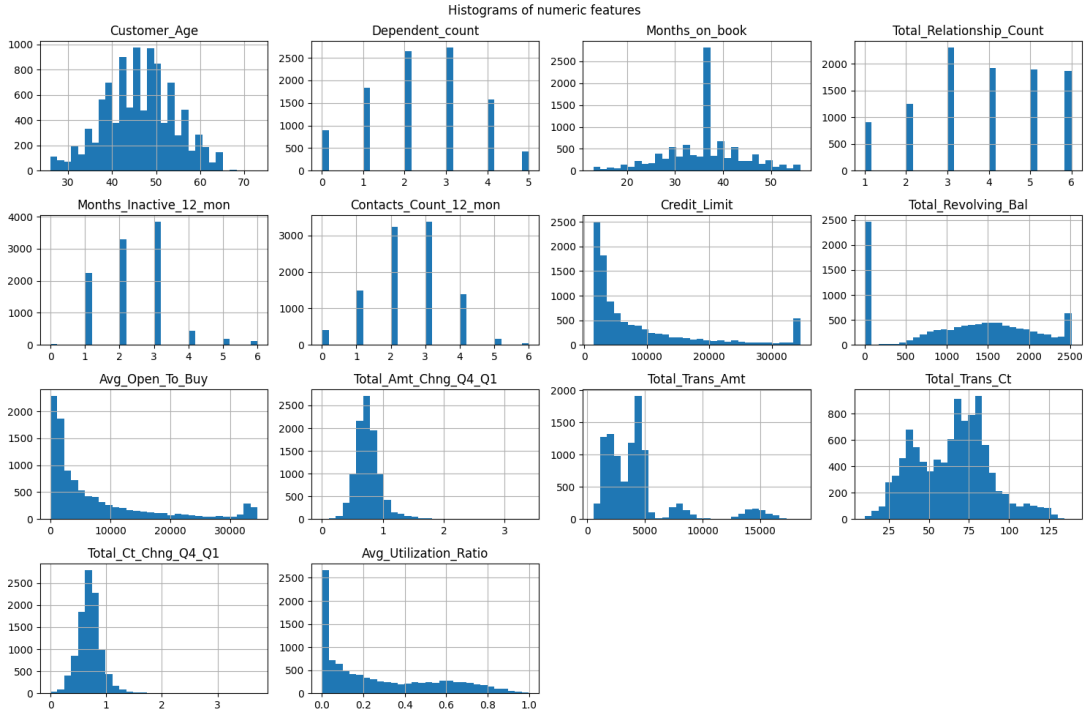

Figure 2: Histograms of the numerical features.

The transaction count and amount distributions are right-skewed, with a heavy tail of very active customers. The credit limit distribution is also right-skewed, reflecting a small minority of premium customers. The change ratios Q4 over Q1 are concentrated around 1, with substantial dispersion on both sides. A ratio equal to 1 means the Q4 value equals the Q1 value (the customer's behaviour did not change between the two quarters), values below 1 indicate a decrease and values above 1 indicate an increase.

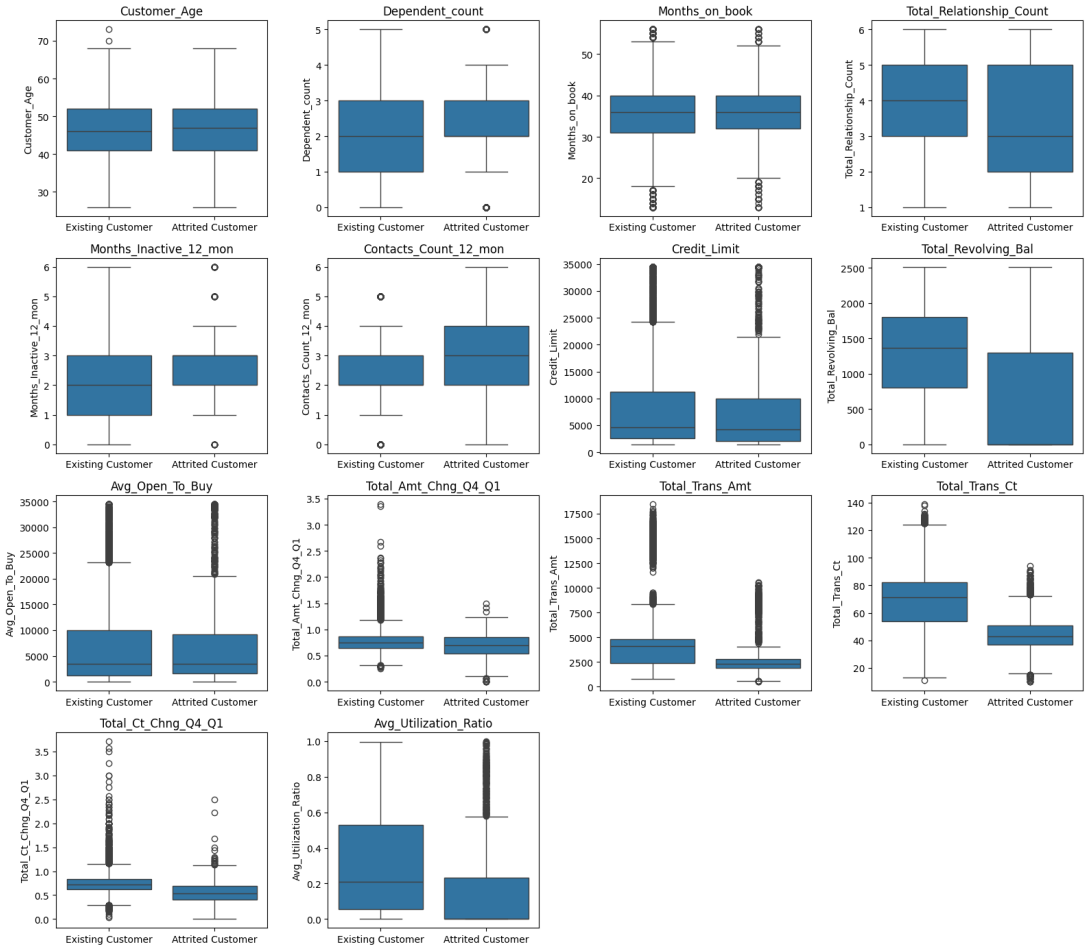

Figure 3: Boxplots of numerical features, split by the target.

The boxplots split by target show the same pattern across features : attrited customers have lower transaction counts, lower transaction amounts, lower change ratios between Q1 and Q4, and higher months of inactivity. These behavioural variables carry most of the signal.

2.3 Categorical features

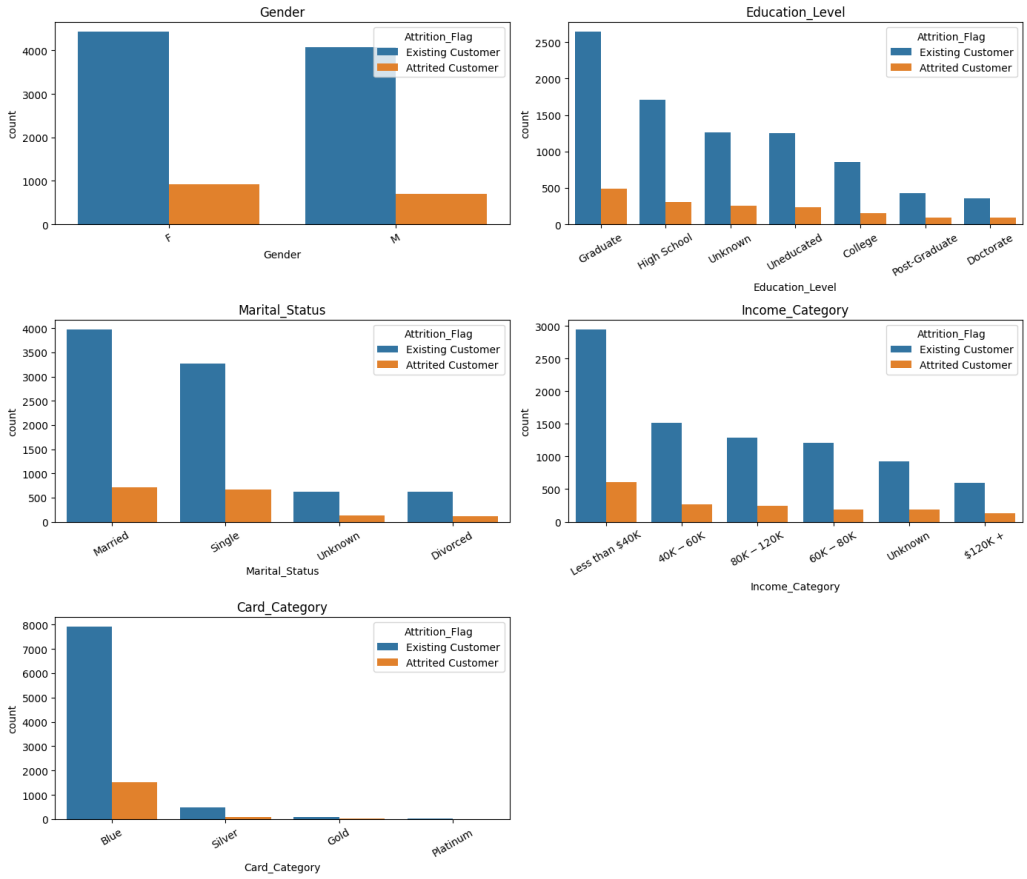

Figure 4: Distribution of the categorical features.

The categorical features have less discriminative power than the behavioural ones. The majority of customers hold the entry-level Blue card, the higher tiers (Gold and Platinum) are rare. I one-hot encode all five categorical features with drop_first=True. The "Unknown" modality, which appears in three of the categorical features, is kept as a regular category since it has no natural ordering.

2.4 Correlations

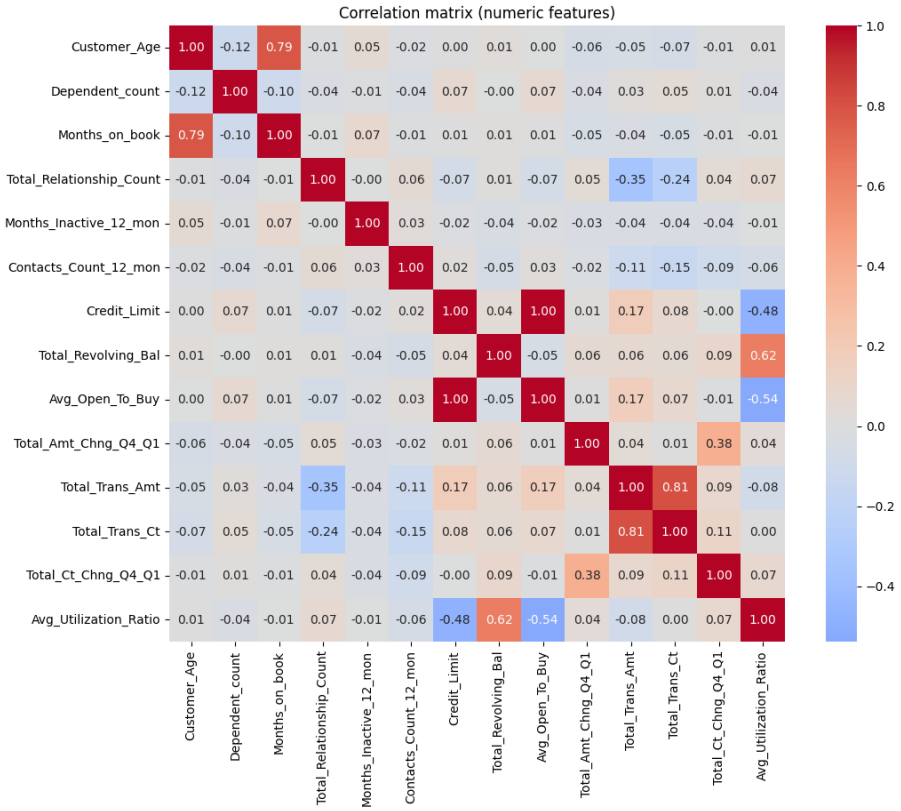

Figure 5: Correlation matrix of the numerical features.

The correlation matrix highlights two strongly correlated pairs :

- `Credit_Limit` and `Avg_Open_To_Buy` : correlation 1 . By definition, Avg_Open_To_Buy = Credit_Limit minus the outstanding balance.
- `Total_Trans_Ct` and `Total_Trans_Amt` : correlation 0.81. Customers who transact often also transact for higher amounts.

These collinearities motivate L1 regularization in Step 1.

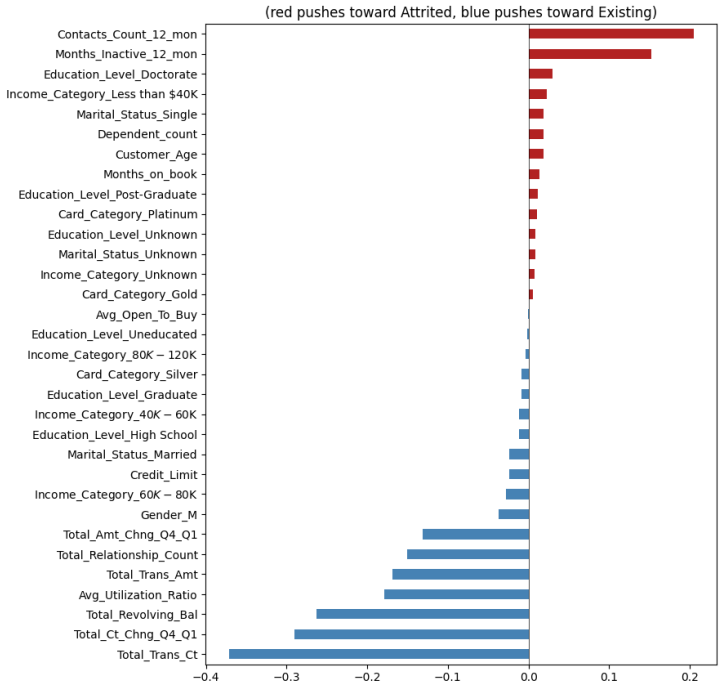

Figure 6: Correlation of each numerical feature with the binary target.

The maximum univariate correlation with the target reaches about 0.37 in absolute value, which to be honest looked low to me at first. But this is expected : univariate correlation only captures the linear dependence between a single feature and the target. The tree-based models trained later can use interactions and non-linear effects that no single correlation can see.

The top features (`Total_Trans_Ct`, `Total_Trans_Amt`, `Total_Revolving_Bal`, `Total_Ct_Chng_Q4_Q1`, `Total_Relationship_Count`) will reappear in every importance method of the modelling sections.

3 Methodology

3.1 Train, validation, test split

I split the 10 127 customers into a training set (70%), a validation set (15%), and a test set (15%), stratified on the target to preserve the 16% / 84% balance in each subset. The random seed is fixed at 42 for reproducibility.

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
)
```

Listing 1: Stratified split with fixed seed.

The validation set is used for model selection between the three approaches and for threshold tuning. The test set is kept untouched until the final evaluation.

3.2 Preprocessing

3.2.1 One-hot encoding

All five categorical features are one-hot encoded with pd.get_dummies(drop_first=True).

3.2.2 Standardization

The 14 numerical features are standardized with StandardScaler. The scaler is fit on the training set only, then applied to validation and test. Fitting on the full data would leak information about the validation and test distributions into the training process.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train), columns=X_train.columns, index=
       X_train.index
)
X_val_scaled = pd.DataFrame(
    scaler.transform(X_val), columns=X_val.columns, index=X_val.index
)
X_test_scaled = pd.DataFrame(
    scaler.transform(X_test), columns=X_test.columns, index=X_test.index
)
```

Listing 2: Scaling without leakage.

I keep two versions of the training matrix :

- X_train_scaled for the Lasso step. L1 regularization penalizes coefficients on the same scale, so unscaled features would be penalized inconsistently.
- X_train_raw for the Random Forest and XGBoost. Tree-based models are scale-invariant : any monotonic transformation of a feature produces the same splits.

3.2.3 Class imbalance

I handle the 16% / 84% imbalance by reweighting samples, not by resampling. For the linear and tree models I set class_weight='balanced'. For XGBoost I set scale_pos_weight to the ratio of negative to positive samples in the training set.

```
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
# scale_pos_weight is approximately 5.22 here
```

Listing 3: Class weighting for XGBoost.

Resampling (SMOTE, random undersampling) is an alternative, but it changes the empirical distribution that the model sees.

3.3 Evaluation metric : AUPR

For all hyperparameter tuning I use the Area Under the Precision-Recall curve (AUPR), implemented as average_precision_score in scikit-learn. AUPR is the standard metric for imbalanced classification :

- It is threshold-independent, like ROC AUC, so it captures the intrinsic ranking quality of the model rather than its performance at an arbitrary cutoff.

- It is more sensitive than ROC AUC to the positive class on imbalanced data. With only 16% positives, ROC AUC can be misleadingly high because the model easily ranks the 84% of negatives correctly.

ROC AUC is still reported alongside AUPR throughout the project for completeness.

3.4 Cross-validation strategy

Hyperparameters are tuned with GridSearchCV using 5-fold stratified cross-validation on the training set. The folds preserve the 16% / 84% balance.

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

Listing 4: Common CV strategy for all three models.

The same cv object is reused for the Random Forest grids and the XGBoost grid, ensuring that the four models see exactly the same data in each fold.

3.5 The three modelling approaches

1. Lasso (LogisticRegressionCV with L1 penalty and solver='liblinear') on the scaled features, used as a feature selector. The features with non-zero coefficients are then fed to a Random Forest with the standard grid.
2. Random Forest on the full raw feature set, with the same grid.
3. XGBoost on the Lasso-reduced raw feature set, with a separate boosting-specific grid.

The three approaches share the same training data, the same cross-validation splits, and the same scoring metric. They differ only in the model class and, for XGBoost, in the hyperparameter grid (learning rate or column subsampling, for example).

4 Step 1 : Lasso Feature Selection

4.1 L1 regularization

Logistic regression with L1 regularization minimizes

$$\min_{\beta_0,\beta} \; -\frac{1}{n}\sum_{i=1}^{n} \log p(y_i \mid x_i; \beta_0, \beta) + \frac{1}{C}\|\beta\|_1 \tag{1}$$

where the L1 norm $\|\beta\|_1 = \sum_j |\beta_j|$ pushes individual coefficients to exactly zero. The hyperparameter C controls the strength of the penalty (large C gives a weak penalty, small C gives a strong one).

4.2 Implementation

I use LogisticRegressionCV, which performs the cross-validation over C internally :

```
from sklearn.linear_model import LogisticRegressionCV

lasso = LogisticRegressionCV(
    Cs=20,
    cv=cv,
    penalty='l1',
```

```
    solver='liblinear',
    scoring='average_precision',
    class_weight='balanced',
    max_iter=5000,
    random_state=SEED,
    n_jobs=-1,
)
lasso.fit(X_train_scaled, y_train)
```

Listing 5: Lasso fit with embedded cross-validation.

Passing Cs=20 to LogisticRegressionCV asks scikit-learn to build a 20-point log-spaced grid for the regularization strength C, then pick the value that maximises validation AUPR through the embedded cross-validation.

4.3 Selected features

The cross-validation selects $C = 0.0336$, which corresponds to a moderately strong regularization. At this value, 21 features out of 32 survive with a non-zero coefficient.

Feature	Coefficient
`Total_Trans_Ct`	-2.5681
`Total_Trans_Amt`	$+1.4011$
`Total_Revolving_Bal`	-0.5660
`Total_Relationship_Count`	-0.5507
`Total_Ct_Chng_Q4_Q1`	-0.5437
`Contacts_Count_12_mon`	$+0.4891$
`Months_Inactive_12_mon`	$+0.4605$
`Gender_M`	-0.2439

Table 1: Top 8 Lasso coefficients (sorted by absolute magnitude, in standardized space).

The dropped features include `Avg_Open_To_Buy` (collinear with `Credit_Limit`, which is also dropped : the two near-identical features are eliminated together at this regularization level), `Customer_Age`, and several demographic dummies.

Most signs make sense in business terms :

- Strong negative coefficients on `Total_Trans_Ct`, `Total_Revolving_Bal`, `Total_Relationship_Count`, and `Total_Ct_Chng_Q4_Q1` : customers who transact often, carry a revolving balance, hold multiple products with the bank, and whose Q4 activity does not collapse versus Q1 are less likely to churn.
- Positive coefficients on `Months_Inactive_12_mon` and `Contacts_Count_12_mon` : inactivity and frequent customer-service interactions are strong predictors of imminent churn.

The positive sign on `Total_Trans_Amt` (+1.40) looked strange to me at first, since high spending should normally protect against churn. The model is actually using the two features for two different things. `Total_Trans_Ct` captures how often the customer transacts, and a high count protects. `Total_Trans_Amt`, once the count is in the model, captures the average ticket size (spend per transaction). A high ticket size means the customer makes a few big purchases rather than many small ones, which looks like an occasional user. Occasional users are less attached to the card, so the model marks them as more at risk. The two signs together are coherent : transaction count protects, but a large average ticket adds risk.

5 Step 2 : Random Forest

5.1 Random Forest on the Lasso-reduced set

A Random Forest is trained on the 21 features kept by the Lasso. The hyperparameters are tuned with a GridSearchCV :

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [200, 500, 800, 1200],
    'max_depth': [None, 10, 20],
    'min_samples_leaf': [5, 10, 20],
}

rf = RandomForestClassifier(class_weight='balanced', random_state=SEED,
    n_jobs=-1)

grid_red = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring='average_precision',
    cv=cv,
    n_jobs=-1,
    verbose=1,
)
grid_red.fit(X_train_red, y_train)
```

Listing 6: Grid search for the Random Forest.

The best cross-validation AUPR is 0.930, with a corresponding validation AUPR of 0.920. The best estimator is re-fit automatically by scikit-learn on the entire training set.

5.2 Splitting criterion : Gini

The Random Forest splits each node using the Gini impurity. Gini measures how mixed the classes are inside a node : it goes from 0 when the node only contains one class (pure) to 0.5 in the binary case when the two classes are present in equal proportion (perfectly mixed). At each node, the algorithm picks the split that gives the biggest drop in Gini between the parent and the children.

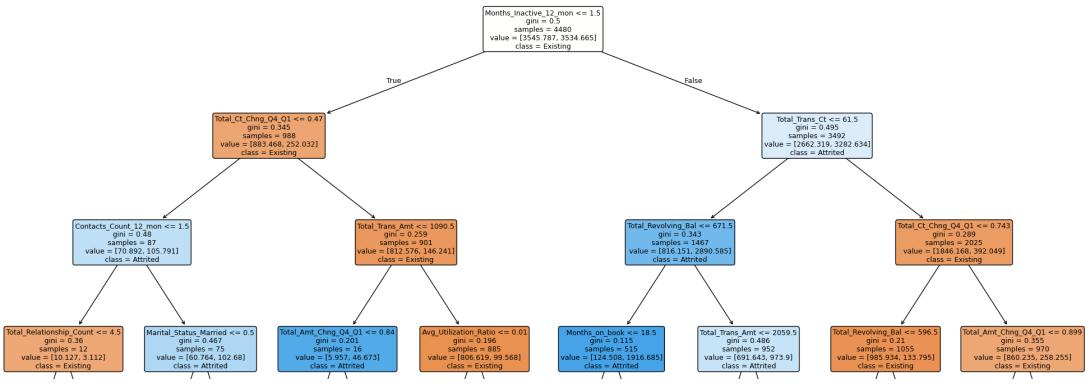

Figure 7: Top three levels of the first tree of the Random Forest reduced. The Gini value, the number of samples, and the weighted class counts are shown at each node.

- The Gini at the root is 0.5, not the value $2 \times 0.16 \times 0.84 = 0.27$ one would compute from raw class proportions. This is because class_weight='balanced' re-weights samples so that the two classes carry equal total weight. The Gini computed on weighted counts reaches its maximum 0.5 at the root.
- The root split here is on `Months_Inactive_12_mon`, not on Total_Trans_Ct (the globally strongest feature). This is because of max_features='sqrt' : at each split, only $\sqrt{21} = 5$ features are randomly drawn out of 21. So different trees pick different roots, and this is what gives the Random Forest its predictive power.

5.3 Mean Decrease Gini

The global feature importance is computed as the average decrease in Gini brought by each feature, weighted by the proportion of samples passing through the corresponding splits, and averaged across all trees of the forest.

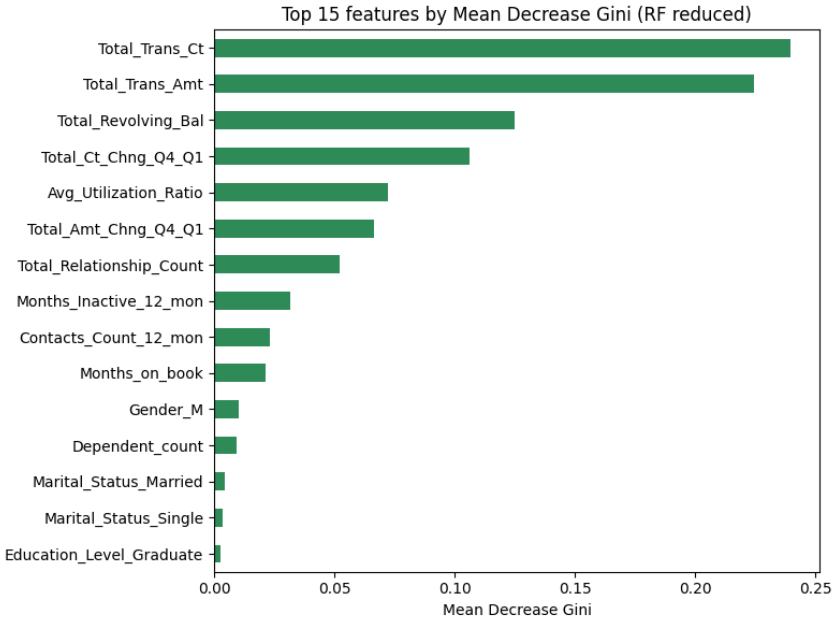

Figure 8: Top 15 features by Mean Decrease Gini in the Random Forest reduced.

The ranking confirms the Lasso ordering : Total_Trans_Ct, Total_Trans_Amt, Total_Revolving_Bal, Total_Relationship_Count, and the two quarterly change ratios sit at the top.

5.4 Random Forest on the full feature set

For comparison, a second Random Forest is trained on the 32 raw features (no Lasso preselection), with the same grid, the same CV, the same scoring, and the same random seed.

	CV AUPR	Val AUPR
RF reduced (21 features)	0.930	0.920
RF full (32 features)	0.926	0.919

Table 2: Random Forest performance with and without Lasso feature selection.

The two Random Forests are basically tied. The Lasso step produced a leaner model (21

features vs. 32 features, with fewer demographic dummies) without really hurting performance, which I find is a good methodological argument for keeping the Lasso step.

6 Step 3 : XGBoost

Gradient-boosted trees fit a sequence of small trees, each one trained to correct the errors of the previous ones. The training minimises the classification loss together with a regularization term that penalises the complexity of each tree.

6.1 Hyperparameter grid

```
import xgboost as xgb
xgb_grid = {
    'n_estimators': [200, 500, 800, 1200],
    'max_depth': [3, 6],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0],
    'min_child_weight': [5, 10],
}

xgb_clf = xgb.XGBClassifier(
    scale_pos_weight=scale_pos_weight,
    random_state=SEED,
    n_jobs=-1,
    eval_metric='aucpr',
)

grid_xgb = GridSearchCV(
    estimator=xgb_clf,
    param_grid=xgb_grid,
    scoring='average_precision',
    cv=cv,
    n_jobs=-1,
    verbose=1,
)
grid_xgb.fit(X_train_red, y_train)
```

Listing 7: XGBoost hyperparameter grid.

The grid differs from the Random Forest one in three ways :

- max_depth=[3, 6] : boosting overfits with deep trees, so I test only shallow ones.
- learning_rate, subsample, colsample_bytree : these are boosting-specific regularizers that have no equivalent in Random Forests.
- scale_pos_weight replaces class_weight='balanced', doing the same job.

The number of CV folds, the scoring metric, the random seed, and the training data are identical to the Random Forest setup.

6.2 Results

The best cross-validation AUPR is 0.966, with a validation AUPR of 0.975. The best n_estimators is 800, which is well inside the grid. I also checked by extending the grid up to 2000 trees, the optimum stays at 800.

	CV AUPR	Val AUPR
RF reduced	0.930	0.920
RF full	0.926	0.919
XGBoost	0.966	0.975

Table 3: Validation performance of the three approaches. XGBoost wins by about four points of AUPR.

6.3 Sanity check : XGBoost on the full feature set

To check whether the Lasso step actually helps XGBoost, I also run a quick training on the 32 raw features (no Lasso preselection), with the same grid. The CV AUPR is 0.972, just above the 0.966 of the Lasso-reduced version. On validation, AUPR moves from 0.975 to 0.977 and F1 stays the same (0.92), but recall drops from 0.95 to 0.93. The gap is small, well within sampling variance.

I keep the 21-feature XGBoost as the main model. The Lasso step does not really help XGBoost in performance, but it does not hurt either, and the model is leaner.

6.4 Confusion matrix at the default threshold

At the default decision threshold of 0.5, the XGBoost classifier produces the following confusion matrix on the validation set :

	Pred. Existing	Pred. Attrited
Actual Existing	1 246	29
Actual Attrited	13	231

Table 4: XGBoost confusion matrix on validation, default threshold = 0.5.

Recall is high (around 0.95) because scale_pos_weight pushes the model to predict the positive class more often. Precision is around 0.89. I tune the threshold in Section 7 and again in Section 9 using a business cost function.

7 Model Selection and Threshold Tuning (F1)

7.1 Validation summary

Comparing the three models on the validation set, XGBoost dominates by a clear margin (Table 3).

7.2 Threshold tuning on F1

The default threshold of 0.5 is somewhat arbitrary. To find a more principled cutoff, I sweep thresholds in $[0.05, 0.95]$ and pick the one that maximises the F1 score on the validation set :

```
y_proba_val = xgb_best.predict_proba(X_val_red)[:, 1]

thresholds = np.arange(0.05, 1.0, 0.05)
rows = []
for t in thresholds:
    yp = (y_proba_val > t).astype(int)
    rows.append({
```

```
        'Threshold': round(t, 2),
        'Accuracy':  accuracy_score(y_val, yp),
        'Recall':    recall_score(y_val, yp),
        'Precision': precision_score(y_val, yp, zero_division=0),
        'F1':        f1_score(y_val, yp, zero_division=0),
    })
threshold_df = pd.DataFrame(rows).round(4)
best_threshold = threshold_df.loc[threshold_df['F1'].idxmax(), '
    Threshold']
```

Listing 8: F1-max threshold search on validation.

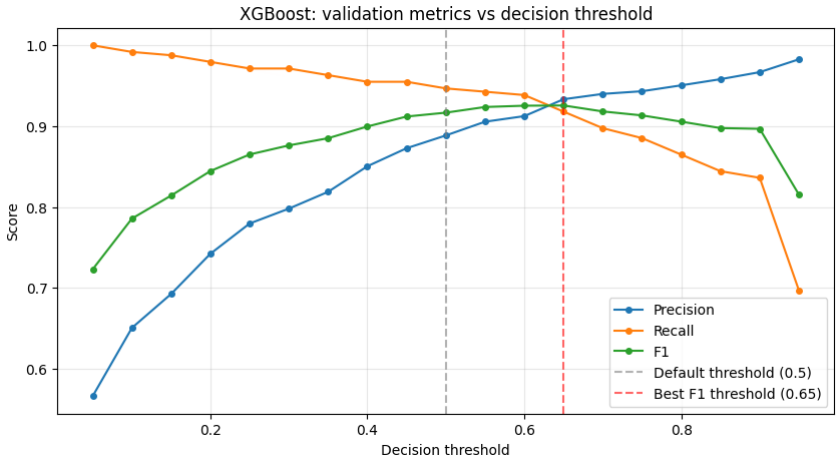

Figure 9: Validation metrics of the XGBoost classifier as a function of the decision threshold. F1 peaks at threshold = 0.65.

F1 peaks at threshold 0.65, well above the default 0.5. At this threshold the model is more balanced between precision and recall than at the default.

8 Interpretability

I use four complementary methods to identify which features drive the XGBoost predictions, and check that they converge on the same short list.

8.1 XGBoost built-in importance

Each tree split assigns a "gain" to the feature it splits on : the reduction in loss obtained by the split. The built-in feature importance averages the gain of each feature across all the splits where it is used.

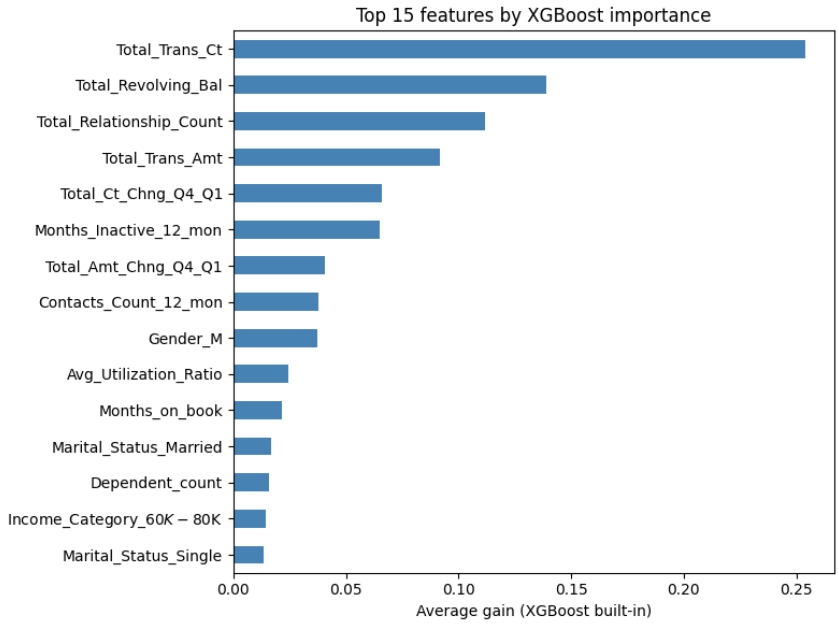

Figure 10: Top 15 features by XGBoost built-in importance.

Total_Trans_Ct, Total_Trans_Amt and Total_Revolving_Bal dominate, followed by the change ratios and Total_Relationship_Count.

8.2 Permutation importance

Permutation importance measures how much the validation AUPR drops when a feature's values are randomly shuffled.

```
from sklearn.inspection import permutation_importance

perm = permutation_importance(
    xgb_best, X_val_red, y_val,
    n_repeats=10,
    random_state=SEED,
    n_jobs=-1,
    scoring='average_precision',
)
```

Listing 9: Permutation importance on the validation set.

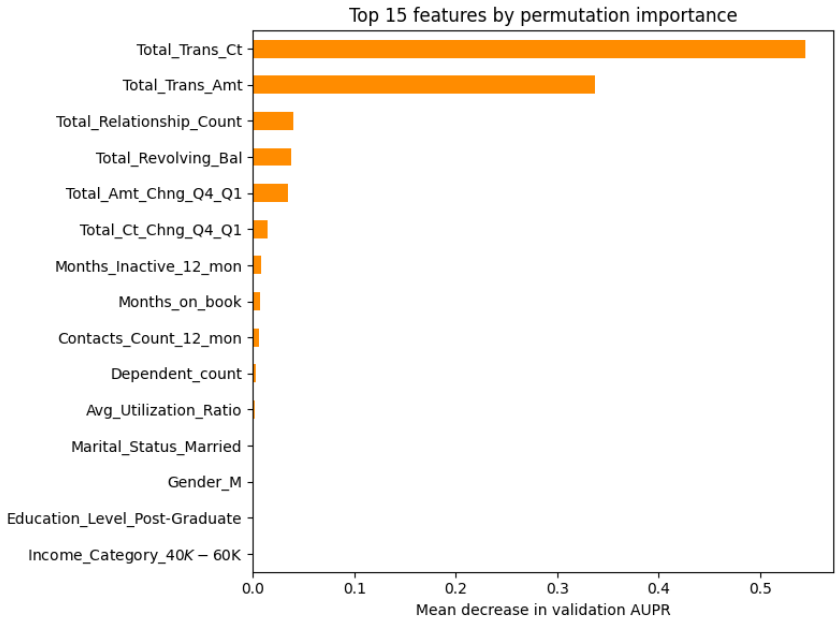

Figure 11: Top 15 features by permutation importance on the validation set.

The same top features appear, with consistent ordering.

8.3 SHAP values

SHAP decomposes each prediction into a sum of contributions, one per feature. For tree-based models, the TreeExplainer computes exact SHAP values efficiently.

```
import shap

explainer = shap.TreeExplainer(xgb_best)
shap_values = explainer(X_val_red)

shap.plots.bar(shap_values, max_display=15, show=False)
shap.plots.beeswarm(shap_values, max_display=15, show=False)
```

Listing 10: SHAP analysis with TreeExplainer.

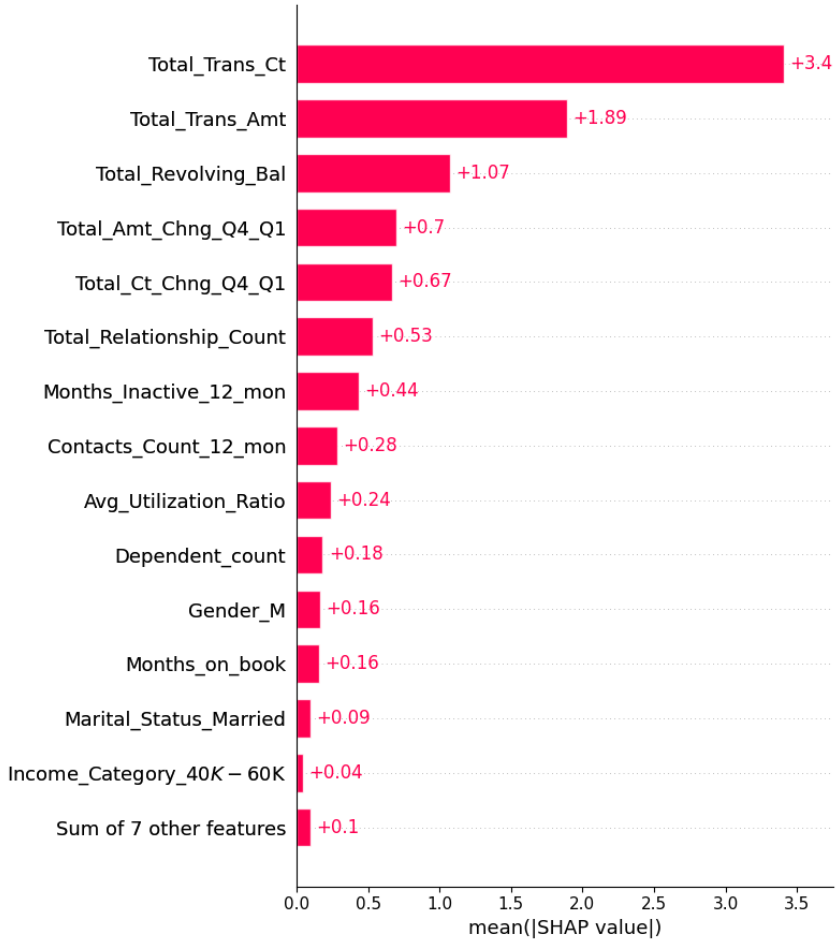

Figure 12: Mean absolute SHAP value of each feature, averaged across the validation set.

The bar plot shows the global magnitude of each feature's contribution. The beeswarm below adds the direction of each effect.

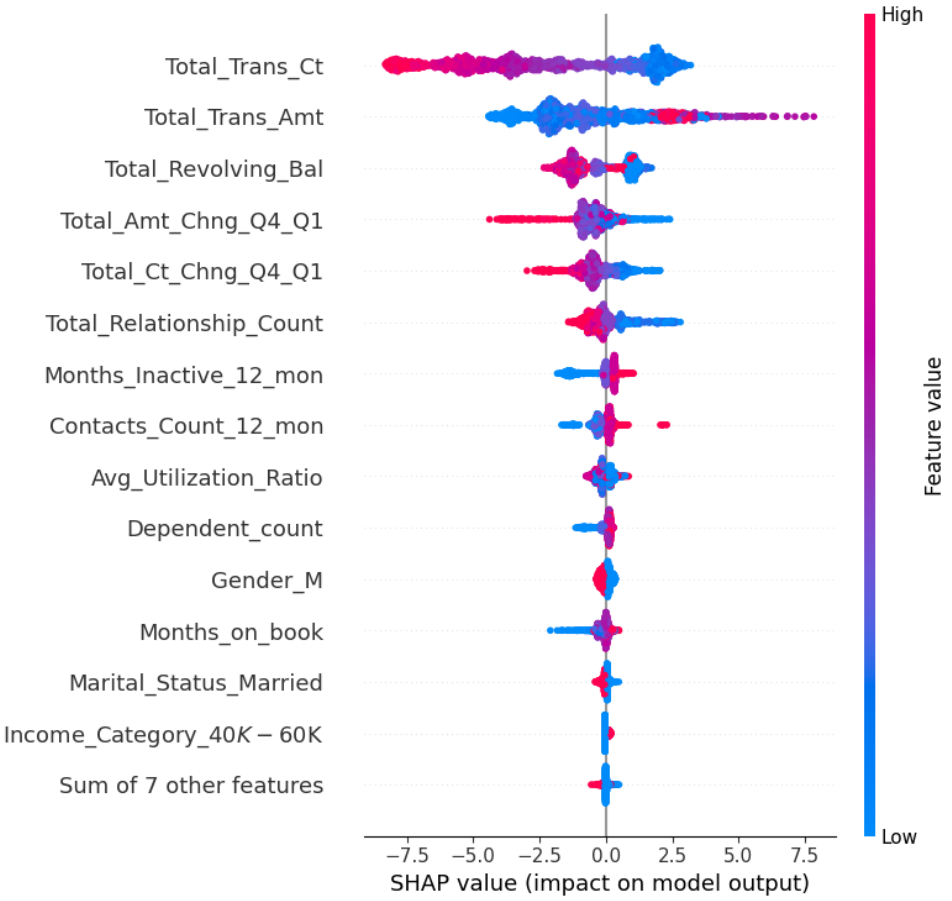

Figure 13: SHAP beeswarm on the validation set. Each dot is one customer, coloured by the feature value (blue low, red high). The horizontal position is the SHAP value.

The dominant features have the widest horizontal spread, which means they have the largest impact on the predictions. The colour gradient on each row goes smoothly from blue to red as the feature value grows, so the model has learned a consistent direction for each of them :

- For Total_Trans_Ct, Total_Revolving_Bal and Total_Relationship_Count, customers with low values are pushed toward churn and customers with high values are protected.
- Total_Trans_Amt works in the opposite direction here : high values push the prediction toward churn. This is the same effect I saw in the Lasso : once the transaction count is in the model, a high amount per transaction looks like an occasional big spender, which the model reads as more at risk.
- Months_Inactive_12_mon and Contacts_Count_12_mon push the prediction toward churn when their values are high.

These directions match the signs of the Lasso coefficients and the business intuition built in the EDA.

8.4 Local explanations

The waterfall plot of SHAP values shows, for one specific customer, how each feature pushed the predicted probability up or down from the base rate :

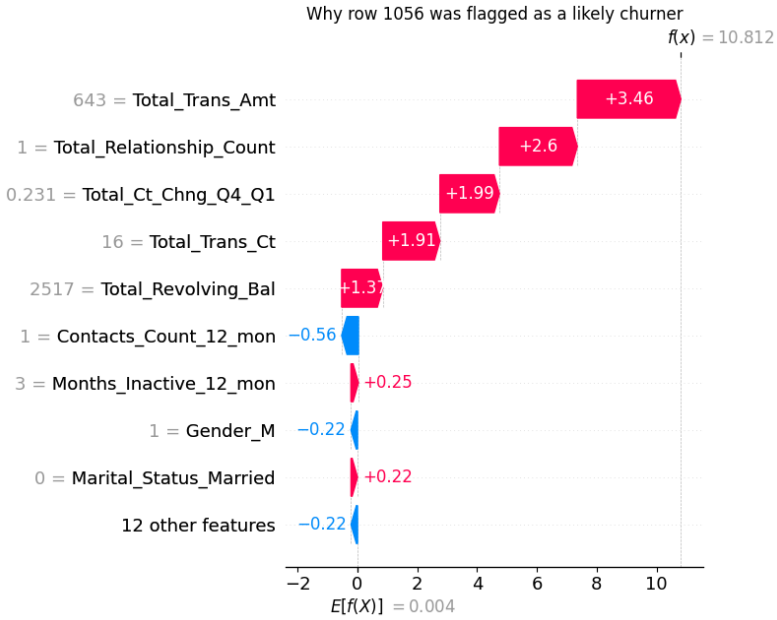

Figure 14: Why customer 1056 was flagged as a likely churner : SHAP waterfall.

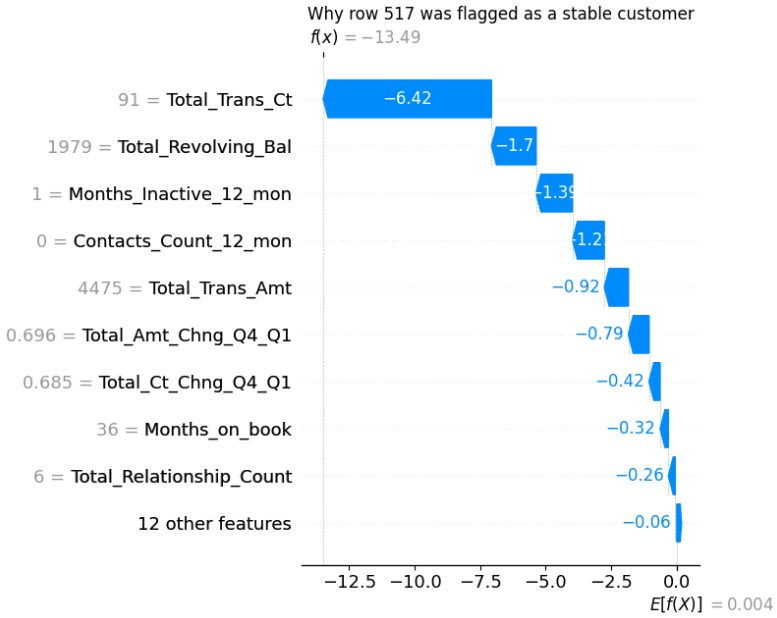

Figure 15: Why customer 517 was flagged as a stable customer : SHAP waterfall.

I think these local explanations are the kind of output you could actually show to a relationship manager who wants to understand why a specific customer was flagged, even if they do not know what SHAP is.

8.5 Convergence of the four views

The four methods rank the same short list at the top : Total_Trans_Ct, Total_Trans_Amt, Total_Revolving_Bal, Total_Relationship_Count, Total_Ct_Chng_Q4_Q1, and Months_Inactive_12_mon. The fact that four different methods give the same ranking gives me confidence in this short list.

9 Business Cost Function

9.1 Why F1 is not enough

The F1-max threshold of 0.65 found in Section 7 treats false positives and false negatives as equally important. In reality they are not :

- A false positive sends a retention offer to a customer who would have stayed anyway. The cost is small.
- A false negative misses a customer who eventually closes their card. The cost is the lifetime value lost, which is much larger.

The right decision threshold should reflect this asymmetry.

9.2 The cost function

I use the following profit function :

$$\text{Profit} = 200 \cdot \text{TP} \; - \; 30 \cdot \text{FP} \; - \; 500 \cdot \text{FN} \tag{2}$$

where :

- 200 $ per TP : net benefit of saving a churner.
- 30 $ per FP : cost of a retention offer sent to a customer who would have stayed anyway.
- 500 $ per FN : cost of a missed churner (estimated lifetime value lost).

9.3 Optimal threshold

I sweep thresholds in $[0.05, 0.95]$ on the validation set, compute the profit at each threshold from the validation confusion matrix, and pick the maximum :

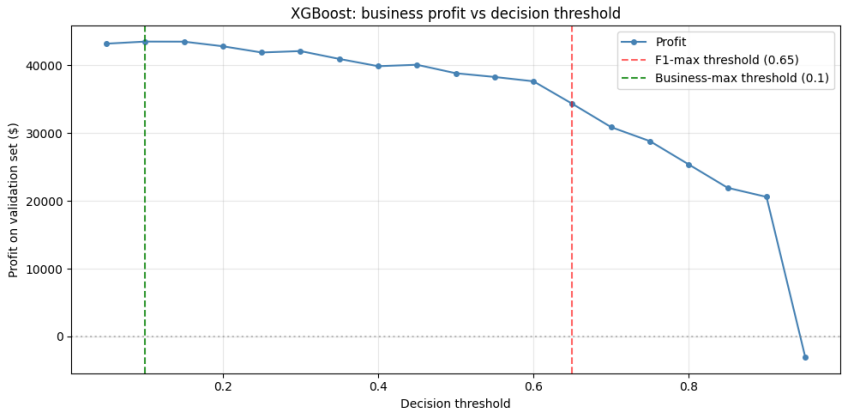

Figure 16: Estimated profit on the validation set as a function of the decision threshold (first-test XGBoost).

The best threshold sits well below the F1 maximum (business threshold = 0.10). Missing a churner costs much more than a false alarm (500 $ vs. 30 $), so the model is pushed to flag a lot of customers as risky. Recall reaches 98% with a precision of 62%, for an expected profit of 41 500 $ on the test set.

10 Second Test : Leakage-Safe Pipeline

Some features in the first model are hard to use before the customer is actually leaving. The bank wants to score customers early enough to intervene, so I rebuild the pipeline after dropping the six features below.

`Total_Ct_Chng_Q4_Q1` and `Total_Amt_Chng_Q4_Q1` : these require a full year of data. By the time I get the data from the last quarter, the customer is already leaving.

`Months_Inactive_12_mon` and `Contacts_Count_12_mon` : by the time these numbers look dangerous, it is already too late to save the customer.

`Total_Trans_Ct` and `Total_Trans_Amt` : low values are confusing. They can mean a customer is about to leave, or simply that the customer naturally uses the service very little. I drop them and build three rate-based ratios in their place (next subsection).

The first test, which uses all six, is the upper-bound benchmark. The second test is the operational version, restricted to features that the bank can act on early.

10.1 Drop the top 3 features, no engineering

I run a first ablation. I drop the three features that every importance method ranks at the top (`Total_Trans_Ct`, `Total_Trans_Amt`, `Total_Revolving_Bal`) and add nothing in their place. The XGBoost grid is the same as the first test.

	CV AUPR	Val AUPR	Test AUPR
First test	0.966	0.975	0.943
Drop top 3	0.773	0.764	0.728

Table 5: Drop the top 3 features, no engineering. Test AUPR loses about 22 points.

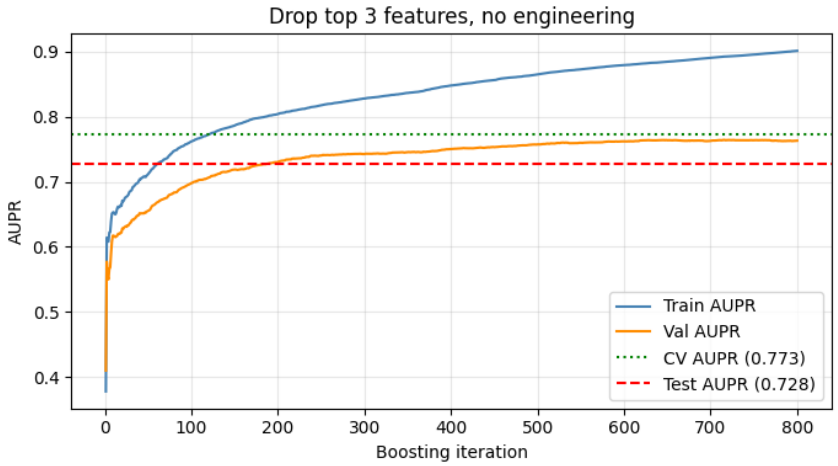

Figure 17: Learning curve when the top 3 features are dropped without any engineering.

Train AUPR climbs to 0.90 while Val plateaus around 0.77, a gap of about 13 points. The model keeps adding trees that fit noise in what is left, since the features that carry most of the signal are gone now.

10.2 Drop the 6 problematic features, no engineering

Now I drop the six features flagged above (the two Q4 / Q1 ratios, `Months_Inactive_12_mon`, `Contacts_Count_12_mon`, `Total_Trans_Ct`, `Total_Trans_Amt`) and again add nothing.

	CV AUPR	Val AUPR	Test AUPR
First test	0.966	0.975	0.943
Drop 6, no engineering	0.488	0.471	0.446

Table 6: Drop the six problematic features, no engineering. Test AUPR drops to about 0.45.

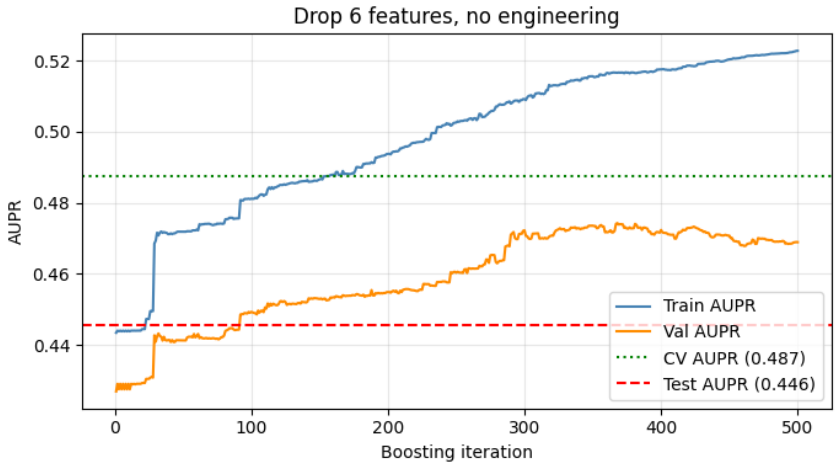

Figure 18: Learning curve when the six problematic features are dropped without any engineering.

Test AUPR is 0.45 with precision 0.34 and F1 0.46. The model is not usable in this state.

10.3 Feature engineering

The three ratios are :

$$\text{Trans\_Ct\_Per\_Month} = \frac{\text{Total\_Trans\_Ct}}{\text{Months\_on\_book}} \tag{3}$$

$$\text{avg\_transaction\_value} = \frac{\text{Total\_Trans\_Amt}}{\text{Total\_Trans\_Ct}} \tag{4}$$

$$\text{spending\_intensity} = \frac{\text{Total\_Trans\_Amt}}{\text{Credit\_Limit}} \tag{5}$$

These ratios use the same raw quantities as the dropped features but recombine them in a way that is meaningful on a rolling window (per-month transaction frequency, average ticket size, and credit usage rate).

10.4 Lasso on the new dataset

I run the Lasso again on the new feature set (29 features after dropping 6 and adding 3). The best C is smaller than in the first test (C = 0.0127), so the regularization is stronger and only 13 features survive out of 29. The strongest coefficient is now `Trans_Ct_Per_Month`, well above any of the original features that stayed. The two other engineered ratios also pass the selection.

Feature	Coefficient
Trans_Ct_Per_Month	−1.9919
Months_on_book	−0.8392
avg_transaction_value	+0.6827
Total_Revolving_Bal	−0.5786
Total_Relationship_Count	−0.4545
spending_intensity	−0.2516
Gender_M	−0.2099
Marital_Status_Married	−0.0937

Table 7: Top 8 Lasso coefficients on the second-test dataset (sorted by absolute magnitude, in standardized space).

10.5 Random Forests on the new dataset

With the same grid, the same CV, and the same seed as in the first test :

	CV AUPR	Val AUPR
RF reduced (second test)	0.842	0.835
RF full (second test)	0.818	0.824

Table 8: Random Forest performance in the leakage-safe second test.

The drop versus the first test is about 9 points of AUPR (the price of removing the temporally leaky aggregates).

10.6 XGBoost on the new dataset

I make the XGBoost grid tighter to avoid overfitting on the weaker engineered features : shallower trees, smaller learning rate, and a larger min_child_weight.

```
xgb_grid = {
    'n_estimators': [200,300,400],
    'max_depth': [2, 3],
    'learning_rate': [0.05, 0.08],
    'subsample': [0.3, 0.5],
    'colsample_bytree': [0.4, 0.6],
    'min_child_weight': [50, 100],
}
```

Listing 11: Tightened XGBoost grid for the second test.

The best CV AUPR is 0.850, with validation AUPR of 0.819.

10.7 Threshold tuning on the second test

The same threshold-tuning procedure is applied to the second-test XGBoost. F1 peaks at threshold 0.70 and the business threshold (using the same 200 / 30 / 500 cost function) sits lower.

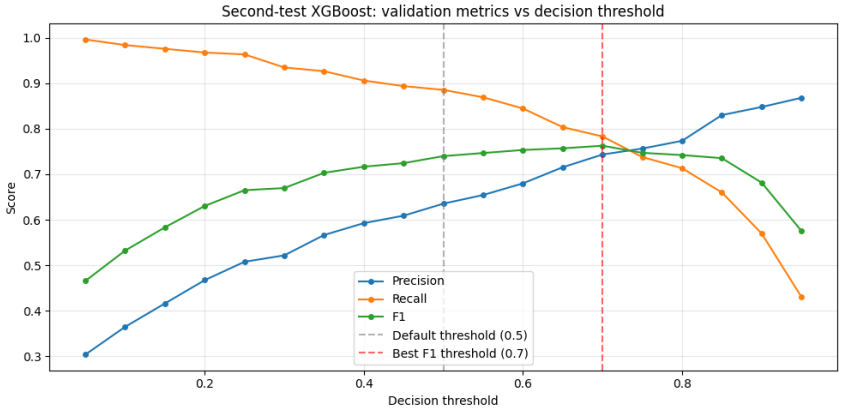

Figure 19: F1, precision and recall as a function of the threshold for the second-test XGBoost.

Figure 20: Business profit as a function of the threshold for the second-test XGBoost.

At its business threshold (t = 0.25), the second model still catches 95% of the churners but at a lower precision (48%).

10.8 Feature importance of the second-test model

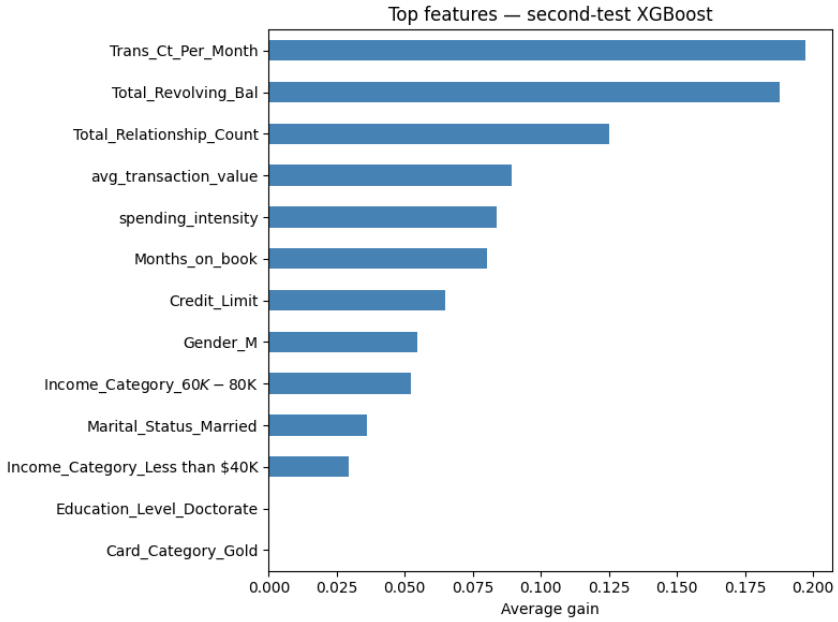

Figure 21: Top features by XGBoost built-in importance in the second test.

The three engineered ratios are at the top, alongside Total_Revolving_Bal and Total_Relationship_Count. The model leans mostly on the ratios I built to replace the dropped features, so the engineering did pick up most of the signal.

11 Final Results on the Test Set

11.1 Test set performance overview

I finally evaluate all six models on the held-out test set. For the XGBoost models I reuse the F1-max thresholds chosen on validation. For the four Random Forests (which were never threshold-tuned) I pick the F1-max threshold on validation here. AUPR and ROC AUC are threshold-independent, so they provide the cleanest comparison.

Model	F1 thr.	AUPR	Recall	Precision	F1	ROC AUC
RF reduced (first test)	0.45	0.919	0.89	0.77	0.83	0.981
RF full (first test)	0.50	0.919	0.86	0.80	0.83	0.980
XGBoost (first test)	0.65	0.943	0.89	0.89	0.89	0.990
RF reduced (second test)	0.45	0.795	0.82	0.66	0.73	0.949
RF full (second test)	0.50	0.766	0.71	0.67	0.69	0.941
XGBoost (second test)	0.70	0.791	0.76	0.70	0.73	0.948

Table 9: Test set performance of the six fitted models. The F1 threshold is picked per model on validation.

11.2 Confusion matrices at the F1-max threshold

	First XGBoost (t = 0.65)			Second XGBoost (t = 0.70)	
	Pred. Existing	Pred. Attrited		Pred. Existing	Pred. Attrited
Actual Existing	1 249	27	Actual Existing	1 197	79
Actual Attrited	28	216	Actual Attrited	58	186
	Recall 0.89, Precision 0.89			Recall 0.76, Precision 0.70	

Table 10: Confusion matrices of the two XGBoost models on the test set, at their respective F1-max thresholds.

The second model misses more churners (FN 58 vs. 28) and produces about three times more false alarms (79 vs. 27).

11.3 Learning curves

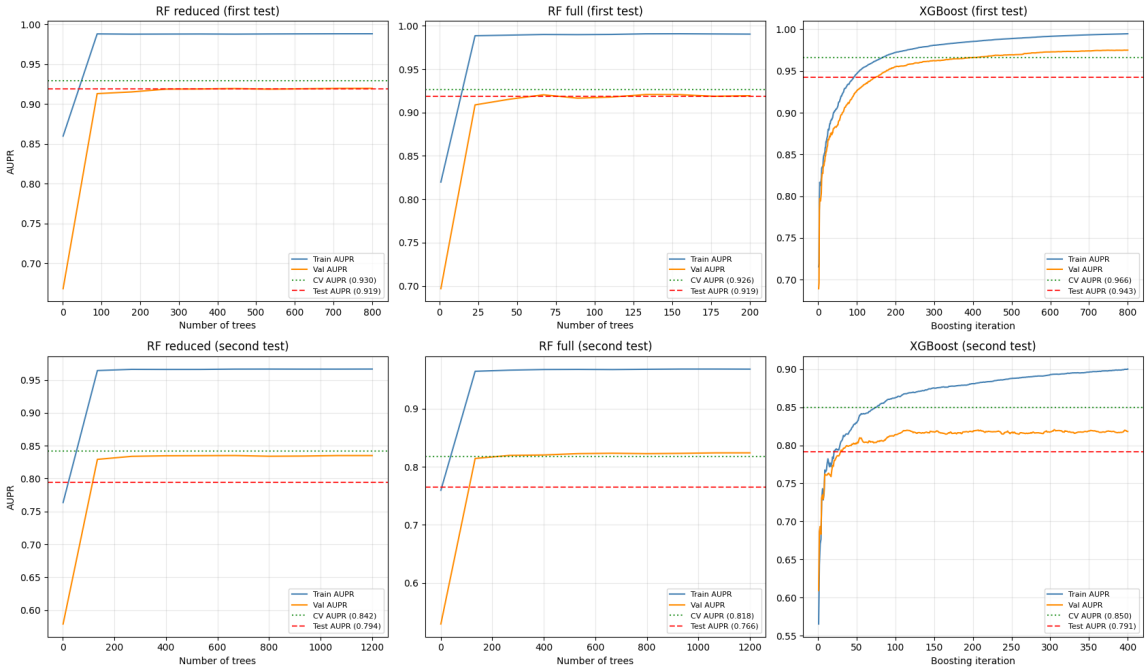

Figure 22: Learning curves of the six models. Top row : first test. Bottom row : second test. Train, validation, cross-validation and test AUPR as a function of the number of trees or boosting iterations.

The top row (first test) shows Val, CV and Test curves on top of each other at around 0.92 for the Random Forests and 0.95 for XGBoost. The hyperparameters I picked on validation generalize well to the test set. Train AUPR is higher (close to 0.99 for the Random Forests and close to 1.0 for XGBoost), but this is just bagged and boosted trees fitting their training data, not overfitting, because Val tracks CV and Test.

The bottom row (second test) shows the start of an overfitting pattern that did not exist in the first test :

- For both Random Forests, the Test curve sits a few points below the Val and CV curves. This gap (around 3 to four points of AUPR) is a sign that the model picked on validation does not generalize as well to the test set as it did on the first test.

- For XGBoost, the Train curve keeps going up after the Val curve has stopped. Train reaches 0.90 and is still rising at the end of the boosting iterations, while Val flattens around 0.82. The model keeps adding trees that fit noise in the training set without helping on validation. The tighter grid (shallow trees, large min_child_weight) reduces this but does not remove it.

The gap is the natural consequence of working with a weaker feature set : the engineered ratios carry less raw signal than Total_Trans_Ct and Total_Trans_Amt, so the model has less room before it starts fitting noise.

11.4 Business profit on the test set

At their respective business thresholds, the two XGBoost models deliver :

- First XGBoost (business threshold = 0.10) : recall 0.98, precision 0.62, expected profit 41 500 $ per evaluation period.
- Second XGBoost (business threshold = 0.25) : recall 0.95, precision 0.48, expected profit 32 750 $ per evaluation period.

The 21% gap between the two figures quantifies the cost of restricting the model to features that can be refreshed in production.

12 Conclusion

12.1 Key findings

1. XGBoost wins both rounds. On the first feature set, it beats the two Random Forests by four points of CV AUPR. On the leakage-safe feature set, it is still the best of the three approaches.
2. The Lasso step is worth keeping. The Random Forest on the Lasso-reduced set gets the same score as the Random Forest on the full set, with only 21 features out of 32. The Lasso gives a smaller model with no loss.
3. Feature importance is stable across methods. Lasso coefficients, XGBoost gain, permutation importance, and SHAP values rank the same short list at the top, which gives confidence in the interpretability of the model when explained to a non-technical audience.
4. The business threshold is much lower than the F1 maximum. Missing a churner costs more than a false alarm, so the cost-driven threshold pushes the model to flag many customers as risky.
5. Switching to the leakage-safe pipeline costs about 21% of profit on the test set. This is the price of using only features that can be refreshed monthly, and at least it gives the bank a concrete number to weigh the trade-off, instead of an abstract argument.

12.2 Recommendation

I recommend using the second XGBoost as the model for the monthly retention campaign, at its business threshold. This comes with a caveat I explain in Section 12.3 : the second pipeline is not fully leakage-free, since the three engineered ratios still use some of the same raw data as the dropped features.

So why prefer the second model over the first ? Because the first model uses 6 features that the bank cannot really use to act early. The Q4/Q1 change ratios only exist once the full year

has passed, and `Months_Inactive_12_mon` and `Contacts_Count_12_mon` fire too late : by the time a customer has been inactive for months or has called customer service several times, the retention window is mostly closed. The second model trades a small leakage residue for features that the bank can update on a rolling basis. The 21% profit gap is the price of that trade.

The first XGBoost should be kept as a benchmark, both to monitor the gap and to watch for drift in the second model.

12.3 Limits and caveats

Non-nested cross-validation. The Lasso picks features using all of X_train. The Random Forest GridSearchCV then does its own cross-validation inside the same X_train, on the features the Lasso has already seen. So each of the five folds is a bit less independent than the CV setup assumes : the features were not chosen blind to the fold split. The CV scores for the Random Forest are a little optimistic compared to a fully nested CV that would re-run the Lasso inside every fold.

Cost values are guesses. The 200 / 30 / 500 dollar figures are round numbers, not real bank data. To make the recommendation more solid, I would run a sensitivity check by varying the missed-churner cost from 200 to 1000 and see if the best threshold changes much.

Capped feature. `Months_Inactive_12_mon` is capped at 6 in the dataset, which probably means there is a filter upstream that I do not see : customers who stay inactive longer may be reclassified as dormant before they reach the model. In production this would need a check.

The "leakage-safe" pipeline is not fully leakage-free. The second test drops six features for the reasons described in Section 10 and adds three rate-based ratios. Two of these ratios (`avg_transaction_value` and `spending_intensity`) are built from `Total_Trans_Ct` and `Total_Trans_Amt`, the very features I just dropped. The ratios reuse the same raw data instead of replacing it : if the totals describe the customer's behaviour during the churn process, their ratio carries the same time signal. The idea is that a rate (per month, per credit dollar) is easier to update on a rolling basis than an aggregate total, but the dataset is a single snapshot, so I cannot really prove this here. A fully leakage-free pipeline would need time-series data with a clear prediction horizon and an out-of-time test. The 21% profit gap should be read as a lower bound on the true cost of leakage-safety, not the final number.